

Capgemini Exceller 2027

Dev / Entry-Level Assessment: Complete Prep Material

Technical and Situational Module | Debugging Assessment | AI-Assisted Coding | Coding Round | English | Cognitive | Behavioural | HR

How to use this PDF. Follow the 14-day plan in Section 2. Every section has theory, worked examples and practice questions with answers. **Honest note on PYQs:** Capgemini rotates its question bank, so exact questions rarely repeat. The questions here are rebuilt from the *patterns* reported by candidates for recent drives (2025-2026). Treat them as a pattern map, not a leaked paper. The exact format can differ by campus and drive.

Contents

- 1. Exam pattern and scoring overview
- 2. 14-day preparation plan
- 3. Technical module A: AI Literacy (notes + MCQs)
- 4. Technical module B: CS fundamentals, cloud and networking (notes + MCQs)
- 5. Output-prediction and pseudocode practice
- 6. Situational awareness: client emails and downtime scenarios
- 7. Debugging assessment: method, bug catalogue, 15 solved snippets
- 8. AI-assisted coding / feature development: workflow, prompt templates, review checklist
- 9. Coding round: patterns and 10 solved problems
- 10. English communication
- 11. Cognitive games and behavioural assessment
- 12. HR and technical interview questions
- 13. Last-48-hours checklist and sources

1. Exam Pattern and Scoring Overview

Reported structure for Exceller 2026/2027 drives (sources: FACE Prep, Placement Preparation, GUVI, Knowledge Gate, candidate threads). Capgemini does not publish one fixed pattern, so confirm in your invite email.

Stage	What it tests	Typical format (reported)
English communication	Grammar, reading, vocabulary, sentence correction	About 30 questions, timed, MCQ
Technical module	Section 1: AI Literacy. Section 2: DSA, CS fundamentals, cloud, networking, situational/workplace scenarios	About 40 MCQs total (roughly 20 + 20), no negative marking reported
Debugging assessment	Find and fix bugs in given C / C++ / Java code (trace and logic bugs)	Several snippets, fix by editing or choosing the corrected line
AI-assisted coding / feature development	Solve a problem or add a small feature by directing an AI assistant step by step	Guided scaffolded task; scored on HOW you work with AI, not only the final answer
Cognitive assessment	Game-based: pattern recognition, memory, attention, logic	About 4 interactive games
Behavioural / power skills	Work-style questionnaire	Answer honestly and consistently
Coding round (some drives)	Two programming problems, test-case based	About 45 minutes, C / C++ / Java / Python
Interviews	Technical + HR	After the online stages

Strategy by section

- **Technical MCQs:** no negative marking is commonly reported, so never leave a question blank. Skip long pseudocode traces first, return later.
- **Debugging:** read the problem statement and expected output FIRST, then hunt for the 6 classic bug types (Section 7).
- **AI-assisted coding:** break the problem into small steps, give the AI context, test, then review the generated code critically. Never paste the whole problem and accept blindly.
- **Proctoring:** assessments are AI-proctored. Keep the camera on, no tab switching, stable internet, ID ready.

2. 14-Day Preparation Plan

Day	Focus	Output
1	Pattern study, AI Literacy notes (Sec 3), 20 MCQs	Notes sheet of key terms
2	OS + DBMS notes (Sec 4), 30 MCQs	Cheat sheet of 20 definitions
3	Networking, cloud, ports, IP/subnet (Sec 4)	Port + IP range table memorised
4	OOP, DSA complexities, sorting/searching	Complexity table memorised
5	Output prediction (Sec 5): 40 snippets	Error log of mistakes
6	Debugging bug catalogue + snippets 1-8 (Sec 7)	Fix each in under 2 minutes
7	Debugging snippets 9-15, then 10 fresh buggy programs from any source	Timed run, 25 minutes
8	AI-assisted coding workflow (Sec 8); practise with any AI assistant on 3 small features	3 prompt chains saved
9	Coding round: arrays and strings (Sec 9)	5 problems solved without looking
10	Coding round: hashing, two pointers, DP basics, matrix	5 problems solved
11	English (Sec 10) + cognitive games practice (Sec 11)	30-question timed English set
12	Situational scenarios (Sec 6) + HR answers (Sec 12)	Spoken answers recorded
13	Full mock: Tech 40 MCQs + 2 debug + 2 coding, all timed	Score and review
14	Revise error log only; sleep early	Checklist (Sec 13)

3. Technical Module A: AI Literacy

The AI section is reported to be concept-based, awareness level: generative AI foundations, prompts, responsible AI. Learn the vocabulary below.

Key notes

Term	Meaning (exam-ready)
Artificial Intelligence	Machines performing tasks that normally need human intelligence (reasoning, perception, language).
Machine Learning	Systems that learn patterns from data instead of being explicitly programmed. Types: supervised (labelled data), unsupervised (unlabelled, finds clusters), reinforcement (reward/penalty).
Deep Learning	ML using multi-layer neural networks. Powers vision, speech and language models.
Generative AI	Models that create new content (text, images, code, audio) from learned patterns.
LLM (Large Language Model)	Neural network trained on huge text corpora to predict the next token. Built on the transformer architecture.
Token	Chunk of text (word or part of a word) the model reads and writes. Limits and costs are measured in tokens.
Context window	Maximum tokens the model can consider at once (prompt + conversation + answer).
Prompt	The instruction or input given to the model. Quality of the prompt strongly affects quality of output.
Zero-shot / One-shot / Few-shot	Prompting with no examples / one example / a few examples in the prompt.
Chain-of-thought	Asking the model to reason step by step; improves multi-step problems.
Hallucination	Model produces confident but false or fabricated information. Always verify facts, code and citations.
Temperature	Randomness control. Low = focused and repeatable, high = creative and varied.
Fine-tuning	Further training a pre-trained model on specific data for a specific task or style.
RAG	Retrieval-Augmented Generation: fetch relevant documents first, then let the model answer using them. Reduces hallucination, adds fresh/private knowledge.
Embedding	Numeric vector representing meaning of text; similar meaning gives nearby vectors. Used in semantic search.
Prompt injection	Malicious text in input or documents that tries to override the model's instructions.
Bias	Systematic unfairness in outputs, usually inherited from skewed training data.
Responsible AI	Fairness, transparency, accountability, privacy, safety, human oversight.
Guardrails	Rules/filters that keep AI outputs safe and on-policy.

Prompt-writing formula (R-C-T-F)

- **Role:** 'You are a senior Java reviewer.'
- **Context:** what the code or project is, constraints, language and version.
- **Task:** one clear action ('find the bug and explain why').
- **Format:** how you want the answer ('table', 'only the corrected function', 'max 5 bullets').

Responsible-AI rules of thumb (frequent answers)

- Never paste confidential client data, passwords or personal data into a public AI tool.
- Always review and test AI output; the human is accountable.
- Disclose AI use where policy requires it.
- Check outputs for bias; use diverse data and human oversight.

Practice MCQs

Q1. A model confidently states a non-existent research paper. This is called:

- (a) Overfitting
- (b) Hallucination
- (c) Tokenisation
- (d) Fine-tuning

Answer: (b) | Confident fabricated output is hallucination.

Q2. Which technique retrieves documents before generating an answer?

- (a) RAG
- (b) GAN
- (c) K-means
- (d) Dropout

Answer: (a) | Retrieval-Augmented Generation.

Q3. Providing 3 examples inside the prompt is called:

- (a) Zero-shot
- (b) Few-shot
- (c) Fine-tuning
- (d) Pre-training

Answer: (b) | Examples in prompt = few-shot prompting.

Q4. Lower temperature generally gives:

- (a) More random output
- (b) More deterministic, focused output
- (c) Longer output
- (d) Faster training

Answer: (b) | Low temperature reduces randomness.

Q5. Best action before using AI-generated code in production:

- (a) Deploy directly
- (b) Review, test and understand it
- (c) Delete comments
- (d) Ask AI to approve it

Answer: (b) | Human review and testing is mandatory.

Q6. Which is a responsible-AI concern?

- (a) Data privacy
- (b) Colour of UI
- (c) Font size
- (d) Screen resolution

Answer: (a) | Privacy, fairness, transparency, safety.

Q7. Which of these is generative AI?

- (a) Spam filter classifying emails
- (b) Model writing a draft email from a prompt
- (c) Sorting algorithm
- (d) SQL JOIN

Answer: (b) | Generative AI creates new content.

Q8. Supervised learning needs:

- (a) Labelled data
- (b) No data
- (c) Only rewards
- (d) Only clusters

Answer: (a) | Input-output labelled pairs.

Q9. Hidden instructions inside a webpage that make an AI agent ignore its rules is:

- (a) Prompt injection
- (b) Overfitting
- (c) Regularisation
- (d) Caching

Answer: (a) | Classic prompt injection.

Q10. A vague prompt like 'make it better' mainly causes:

- (a) Faster results
- (b) Ambiguous, low-quality output
- (c) Encryption
- (d) Compilation errors

Answer: (b) | Clear context and constraints are needed.

Q11. What does a larger context window allow?

- (a) Model to consider more text at once
- (b) Smaller model size
- (c) Free usage
- (d) No hallucination

Answer: (a) | More tokens in view.

Q12. Transformer architecture is most associated with:

- (a) LLMs
- (b) Relational databases
- (c) Compilers
- (d) Routers

Answer: (a) | Modern LLMs are transformer-based.

4. Technical Module B: CS Fundamentals, Cloud and Networking

4.1 Data structures and complexity

Item	Best / Avg / Worst	Notes
Linear search	$O(1)$ / $O(n)$ / $O(n)$	Works on unsorted data
Binary search	$O(1)$ / $O(\log n)$ / $O(\log n)$	Needs sorted data
Bubble / Insertion / Selection sort	$O(n)$ / $O(n^2)$ / $O(n^2)$	Bubble and insertion best case $O(n)$ when already sorted; selection always $O(n^2)$
Merge sort	$O(n \log n)$ all cases	Stable, $O(n)$ extra space
Quick sort	$O(n \log n)$ / $O(n \log n)$ / $O(n^2)$	In-place, not stable
Heap sort	$O(n \log n)$ all cases	In-place, not stable
Hash table lookup	$O(1)$ avg / $O(n)$ worst	Collisions: chaining or open addressing
BST search	$O(\log n)$ avg / $O(n)$ worst	Worst when skewed
Stack / Queue push-pop	$O(1)$	Stack LIFO, Queue FIFO
BFS / DFS	$O(V + E)$	BFS uses queue, DFS uses stack/recursion

4.2 Operating systems

- **Process vs thread:** process has its own memory space; threads share the process memory and are lighter.
- **Deadlock needs all 4:** mutual exclusion, hold and wait, no pre-emption, circular wait.
- **Scheduling:** FCFS, SJF (min avg waiting time), Round Robin (time quantum), Priority (starvation, fixed by ageing).
- **Paging** removes external fragmentation; **segmentation** follows logical program units. **Virtual memory** lets programs exceed RAM; **thrashing** = excessive page faults.
- **Semaphore:** counter for synchronisation; **mutex:** lock owned by one thread.

4.3 DBMS and SQL

- **ACID:** Atomicity, Consistency, Isolation, Durability.
- **Keys:** primary (unique, not null), candidate, foreign (references another table), composite.
- **Normal forms:** 1NF atomic values; 2NF no partial dependency; 3NF no transitive dependency; BCNF every determinant is a candidate key.
- **DELETE** removes rows (can WHERE, rollback-able); **TRUNCATE** removes all rows fast; **DROP** removes the table itself.
- **WHERE** filters rows before grouping; **HAVING** filters groups after GROUP BY.
- **JOINS:** INNER (matching only), LEFT (all left + matches), RIGHT, FULL OUTER, CROSS (cartesian).

```
SELECT dept, COUNT(*) AS emp_count, AVG(salary) AS avg_sal
FROM employee
WHERE salary > 20000
GROUP BY dept
HAVING COUNT(*) > 3
ORDER BY avg_sal DESC;
```

```
-- 2nd highest salary
SELECT MAX(salary) FROM employee
WHERE salary < (SELECT MAX(salary) FROM employee);
```

4.4 OOP

- **Four pillars:** Encapsulation, Abstraction, Inheritance, Polymorphism.

- **Overloading** = same name, different parameters (compile time). **Overriding** = subclass redefines parent method (run time).
- **Java**: no multiple inheritance of classes (use interfaces); **abstract class** can have state and constructors, **interface** defines a contract; **final** prevents override/reassignment; String is immutable.
- **C++**: virtual functions give run-time polymorphism; destructor of a base class should be virtual.

4.5 Networking and cloud

Topic	Quick facts
OSI layers (top to bottom)	Application, Presentation, Session, Transport, Network, Data Link, Physical. Router = Network layer, Switch = Data Link, TCP/UDP = Transport.
TCP vs UDP	TCP: connection-oriented, reliable, ordered (web, email, SSH). UDP: connectionless, fast, no guarantee (DNS queries, streaming, VoIP).
Common ports	HTTP 80, HTTPS 443, FTP 21, SSH 22, Telnet 23, SMTP 25, DNS 53, DHCP 67/68, POP3 110, IMAP 143, RDP 3389, MySQL 3306, PostgreSQL 5432.
Private IPv4 ranges	10.0.0.0/8, 172.16.0.0/12 (172.16 to 172.31), 192.168.0.0/16.
Subnetting	/24 = 256 addresses (254 usable). /25 = 128 (126 usable). /26 = 64 (62). /30 = 4 (2 usable). Usable = $2^{\text{host_bits}} - 2$.
HTTP codes	200 OK, 201 Created, 301 Moved permanently, 400 Bad request, 401 Unauthorised, 403 Forbidden, 404 Not found, 500 Internal server error, 502 Bad gateway, 503 Service unavailable.
DNS / DHCP / NAT	DNS: name to IP. DHCP: auto-assigns IP. NAT: maps private to public IPs.
Cloud models	IaaS (EC2, Azure VM), PaaS (Elastic Beanstalk, App Service), SaaS (Gmail, Salesforce).
Cloud basics	VPC = isolated virtual network; Security group = instance-level firewall (stateful); Load balancer spreads traffic; Auto scaling adds/removes instances; CDN caches content near users; Region = geographic area, Availability Zone = isolated data centre in region.
Deployment models	Public, Private, Hybrid, Multi-cloud.
High availability	Redundancy, multi-AZ, health checks, failover, backups, RTO (time to recover) and RPO (data loss tolerated).

4.6 SDLC, Agile, Git

- **SDLC phases**: requirements, design, development, testing, deployment, maintenance. **Waterfall** = sequential; **Agile** = iterative sprints.
- **Scrum roles**: Product Owner, Scrum Master, Development Team. Events: sprint planning, daily stand-up, review, retrospective.
- **Testing**: unit, integration, system, UAT; black-box (no code knowledge) vs white-box.
- **Git**: clone, add, commit, push, pull, fetch, branch, merge, rebase, stash, revert, reset, log, diff. **fetch** downloads only; **pull** = fetch + merge.
- **CI/CD**: continuous integration builds and tests every commit; continuous delivery/deployment automates release.

Practice MCQs (technical pattern)

Q13. Time complexity of binary search on n sorted elements:

- (a) $O(n)$
- (b) $O(\log n)$
- (c) $O(n \log n)$
- (d) $O(1)$

Answer: (b) | Search space halves every step.

Q14. Which data structure is used for BFS?

- (a) Stack
- (b) Queue
- (c) Heap
- (d) Tree

Answer: (b) | BFS explores level by level with a queue.

Q15. Which SQL command removes the table structure as well?

- (a) DELETE
- (b) TRUNCATE
- (c) DROP
- (d) REMOVE

Answer: (c) | DROP removes the table definition.

Q16. Which layer is responsible for routing?

- (a) Data Link
- (b) Network
- (c) Transport
- (d) Session

Answer: (b) | Routers work at the Network layer.

Q17. Which port does HTTPS use by default?

- (a) 80
- (b) 443
- (c) 22
- (d) 53

Answer: (b) | 443.

Q18. Usable hosts in a /26 subnet:

- (a) 64
- (b) 62
- (c) 30
- (d) 126

Answer: (b) | $2^6 - 2 = 62$.

Q19. Which is NOT a necessary condition for deadlock?

- (a) Mutual exclusion
- (b) Hold and wait
- (c) Pre-emption
- (d) Circular wait

Answer: (c) | No pre-emption is required; pre-emption itself is not.

Q20. Gmail is an example of:

- (a) IaaS
- (b) PaaS
- (c) SaaS
- (d) DaaS

Answer: (c) | Software delivered as a service.

Q21. HTTP status 503 means:

- (a) Not found
- (b) Forbidden
- (c) Service unavailable
- (d) Created

Answer: (c) | Server temporarily cannot handle the request.

Q22. Which sort has worst case $O(n \log n)$?

- (a) Quick sort
- (b) Merge sort
- (c) Bubble sort
- (d) Selection sort

Answer: (b) | Merge sort is always $O(n \log n)$.

Q23. Which clause filters groups?

- (a) WHERE
- (b) HAVING
- (c) ORDER BY
- (d) LIMIT

Answer: (b) | HAVING works after GROUP BY.

Q24. Method overriding is an example of:

- (a) Compile-time polymorphism
- (b) Run-time polymorphism
- (c) Encapsulation
- (d) Abstraction

Answer: (b) | Resolved at run time by the actual object type.

Q25. A load balancer primarily provides:

- (a) Data encryption at rest
- (b) Traffic distribution and availability
- (c) Name resolution
- (d) Code compilation

Answer: (b) | Spreads requests across servers.

Q26. Which protocol is connection-oriented?

- (a) UDP
- (b) TCP
- (c) IP
- (d) ICMP

Answer: (b) | TCP uses a 3-way handshake.

Q27. In Git, which command downloads changes WITHOUT merging?

- (a) git pull
- (b) git fetch
- (c) git push
- (d) git merge

Answer: (b) | fetch updates remote refs only.

Q28. Normal form that removes transitive dependency:

- (a) 1NF
- (b) 2NF
- (c) 3NF
- (d) None

Answer: (c) | 3NF.

Q29. Process in Ready state waits for:

- (a) I/O
- (b) CPU
- (c) Memory dump
- (d) Network

Answer: (b) | Ready = waiting for CPU allocation.

Q30. Which is a stateless protocol?

- (a) HTTP
- (b) FTP control with sessions
- (c) Telnet
- (d) TCP

Answer: (a) | HTTP does not retain state between requests.

Q31. RTO in disaster recovery stands for:

- (a) Recovery Time Objective
- (b) Real Time Output
- (c) Retry Time Option
- (d) Redundant Tier Object

Answer: (a) | Maximum acceptable downtime.

Q32. Which structure follows LIFO?

- (a) Queue
- (b) Stack
- (c) Array
- (d) Graph

Answer: (b) | Last in first out.

5. Output Prediction and Pseudocode Practice

Trace carefully on paper. Answers are at the end of the block.

- 1) Java: `System.out.println(10 + 20 + "Hello" + 10 + 20);`
- 2) Java: `String a = "hi"; String b = new String("hi");
System.out.println((a == b) + " " + a.equals(b));`
- 3) Java: `char c = 'A'; c += 1; System.out.println(c);`
- 4) Java: `System.out.println('A' + 1);`
- 5) Java: `int x = Integer.MAX_VALUE; System.out.println(x + 1);`
- 6) Python: `a = [1,2,3]; b = a; b.append(4); print(a)`
- 7) Python: `print(3 * "ab", 7 // 2, 7 % 3, -7 // 2)`
- 8) C: `int i = 0; while (i < 3) { i++; if (i == 2) continue; printf("%d ", i); }`
- 9) C: `printf("%f", 7 / 2 * 2.0);`
- 10) C: `int a = 5, b = 2; printf("%d %d", a / b, a % b);`
- 11) Python: `def f(x, l=[]): l.append(x); return l
print(f(1), f(2))`
- 12) C: `for (int i = 1; i <= 3; i++) for (int j = 1; j <= i; j++) printf("*");`
- 13) Pseudo: `n=5, s=0; while n>0: s=s+n%2; n=n-1; print s`
- 14) C: `int x = 10; if (x > 5) if (x > 20) printf("A"); else printf("B");`
- 15) Python: `print(bool([]), bool("0"), bool(0))`

Answers:

- | | | | |
|---|-----------------|-------------------------|-------|
| 1) 30Hello1020 | 2) false true | 3) B | 4) 66 |
| 5) -2147483648 | 6) [1, 2, 3, 4] | 7) ababab 3 1 -4 | |
| 8) 1 3 | 9) 6.000000 | 10) 2 1 | |
| 11) [1, 2] [1, 2] (same default list reused!) | | | |
| 12) 6 stars total (1 + 2 + 3) | | 13) 3 (n=5,3,1 are odd) | |
| 14) B (else binds to nearest if) | | 15) False True False | |

Pseudocode tips

- Write a trace table (variable columns, one row per iteration). Do not trace in your head.
- Check loop boundaries (< vs <=), integer division, operator precedence, post vs pre increment.
- Watch for the dangling-else, missing break in switch, and shadowed variables.

6. Situational Awareness: Client Emails and Downtime

Reported scenarios: handling a critical enterprise client email, resolving production downtime, prioritising tasks. The answer pattern is almost always the same.

The 6-step incident framework

Step	What to do
1. Acknowledge	Reply quickly; tell the client you are on it. Do not blame.
2. Assess	Check impact, scope, start time, recent changes, logs and monitoring.
3. Escalate	Inform your lead or on-call engineer. Raise severity by business impact.
4. Communicate	Share regular, honest updates with next-update time. No false promises or ETAs you cannot meet.
5. Resolve	Apply the safest fix or rollback first; restore service before perfecting.
6. Review	Root cause analysis, preventive actions, document lessons.

Practice scenarios

Q33. A key client emails that the portal is down and they are losing sales. Best FIRST action?

- (a) Ignore until the manager returns
- (b) Acknowledge, check impact and escalate to the right team
- (c) Tell the client it is their network problem
- (d) Reboot every server at once

Answer: (b) | Acknowledge, assess, escalate. Never blame or act blindly.

Q34. You find the outage was caused by your own deployment. You should:

- (a) Hide it and quietly fix
- (b) Inform the team immediately, roll back and document
- (c) Wait for someone to notice
- (d) Blame the testers

Answer: (b) | Transparency and fast rollback.

Q35. Two urgent tasks arrive: a minor UI typo for a client and a production failure. Prioritise:

- (a) Typo
- (b) Production failure
- (c) Whichever came first
- (d) Neither

Answer: (b) | Prioritise by business impact and severity.

Q36. Client asks for a feature not in scope and the deadline is tight. Best response:

- (a) Say yes to please them
- (b) Say no and end the call
- (c) Discuss impact, offer options (phasing, change request) with your manager
- (d) Add it silently

Answer: (c) | Manage expectations through proper channels.

Q37. An angry client email demands an immediate fix. Your reply should be:

- (a) Defensive
- (b) Calm, empathetic, factual with next steps and update time
- (c) Very long with technical jargon
- (d) Delayed until fixed

Answer: (b) | Empathy plus clarity.

Q38. A teammate keeps missing deadlines affecting you. First step:

- (a) Complain to the client
- (b) Speak to them privately to understand the issue, then involve the lead if needed
- (c) Do their work forever
- (d) Ignore

Answer: (b) | Direct, respectful communication first.

Template: client downtime reply

Subject: [Incident] Portal outage - we are investigating

Hi <Name>,
Thank you for flagging this. We have confirmed that the portal is currently unavailable and our engineers are actively investigating.
Impact: login and checkout are affected since 10:40 AM IST.
Next steps: we are checking recent changes and server health.
I will send you the next update by 11:30 AM IST, or earlier if service is restored.
We apologise for the disruption.

Regards,
<Your name>

7. Debugging Assessment

You receive working-looking but broken code and must fix it. Languages reported: C, C++, Java. This is a code-reading skill, so practise reading, not writing.

The 5-step method

- **1. Read the problem statement** and the expected output or sample test.
- **2. Run the code mentally** with the sample input using a trace table.
- **3. Check the 6 bug families** below.
- **4. Change the minimum possible;** do not rewrite working code.
- **5. Re-test** against the sample, an edge case (empty, one element, negative, zero) and a large input.

Bug catalogue (check in this order)

Family	Examples
Off-by-one / boundary	$i \leq n$ instead of $i < n$; mid calculation; loop starts at 1 vs 0; array size n vs $n+1$
Initialisation	Uninitialised sum/max/min; max initialised to 0 for negative arrays; counters not reset in loops
Operators	$=$ vs $==$; $\&$ vs $\&\&$; integer division; $\%$ with negatives; precedence; post vs pre increment
Control flow	Missing break; missing $i++$; wrong loop condition; unreachable code; wrong return placement (inside loop)
Types / memory	Integer overflow (use long); char vs int; float compare; missing $\&$ in scanf; string $==$ vs equals (Java); NULL pointer; array out of bounds; missing free; strlen not including $\backslash 0$
Logic	Wrong formula; swapped arguments; pass-by-value swap; recursion without base case; modifying list while iterating; wrong condition in nested if

15 solved snippets

1. Sum of array (C) - expected sum of n elements

```
int sum(int a[], int n) {
    int s; // BUG: uninitialised
    for (int i = 0; i <= n; i++) // BUG: <= goes out of bounds
        s += a[i];
    return s;
}
// FIX: int s = 0; and i < n
```

2. Assignment in condition (C)

```
if (x = 5) printf("five"); // BUG: assigns 5, always true
// FIX: if (x == 5)
```

3. String compare (Java)

```
String a = new String("java"), b = new String("java");
if (a == b) System.out.println("same"); // BUG: compares references
// FIX: if (a.equals(b))
```

4. Missing break (C/Java)

```
switch (d) {
    case 1: printf("Mon"); // BUG: falls through to case 2
    case 2: printf("Tue"); break;
}
// FIX: add 'break;' after case 1
```

5. Integer division (C)

```
int sum = 7, n = 2;
float avg = sum / n; // BUG: gives 3.0
// FIX: float avg = (float) sum / n; -> 3.5
```

6. Infinite loop (C)

```
int i = 0;
while (i < 10) { printf("%d ", i); } // BUG: i never changes
// FIX: add i++ inside the loop
```

7. scanf (C)

```
int n; scanf("%d", n); // BUG: needs address
// FIX: scanf("%d", &n);
```

8. Binary search (Java)

```
int lo = 0, hi = n - 1;
while (lo < hi) { // BUG: misses single remaining element
    int mid = (lo + hi) / 2; // BUG: can overflow for huge values
    if (a[mid] == key) return mid;
    else if (a[mid] < key) lo = mid; // BUG: infinite loop, must be mid+1
    else hi = mid; // BUG: must be mid-1
}
return -1;
// FIX: while (lo <= hi); mid = lo + (hi - lo) / 2; lo = mid + 1; hi = mid - 1;
```

9. Factorial recursion (C)

```
int fact(int n) { return n * fact(n - 1); } // BUG: no base case
// FIX: if (n <= 1) return 1;
```

10. Swap function (C)

```
void swap(int a, int b) { int t = a; a = b; b = t; } // BUG: pass by value
// FIX: void swap(int *a, int *b) { int t = *a; *a = *b; *b = t; }
// call as swap(&x, &y);
```

11. Prime check (Java)

```
boolean isPrime(int n) {
    for (int i = 2; i <= n; i++) // BUG: i reaches n, n % n == 0
        if (n % i == 0) return false;
    return true; // BUG: 0 and 1 reported prime
}
// FIX: if (n < 2) return false; for (int i = 2; (long) i * i <= n; i++)
```

12. Max of array with negatives (C)

```
int max = 0; // BUG: wrong if all numbers negative
for (int i = 0; i < n; i++) if (a[i] > max) max = a[i];
// FIX: int max = a[0]; (loop from i = 1)
```

13. Bubble sort bound (Java)

```
for (int i = 0; i < n - 1; i++)
    for (int j = 0; j < n - i; j++) // BUG: a[j+1] goes out of bounds
        if (a[j] > a[j+1]) { int t = a[j]; a[j] = a[j+1]; a[j+1] = t; }
// FIX: j < n - i - 1
```

14. NullPointerException (Java)

```
String s = null;
if (s.equals("abc")) { ... } // BUG: NPE
// FIX: if ("abc".equals(s)) or check s != null first
```

15. Return inside loop (C)

```
int isPrimeLike(int n) {
    for (int i = 2; i < n; i++) {
        if (n % i == 0) return 0;
        else return 1; // BUG: returns after first iteration
    }
}
// FIX: return 1 only AFTER the loop finishes (and handle n < 2)
```

Speed tips

- Spend about 4-5 minutes per snippet. If stuck, run the sample by hand.
- Fix the root cause once; do not patch symptoms.
- Re-read the question: sometimes the bug is that the code solves a different problem than asked.

8. AI-Assisted Coding / Feature Development

This round scores **how you work with an AI assistant**: framing, step-by-step direction, reviewing and testing. Reported flow: a guided, scaffolded task where you build a feature in stages.

What evaluators look for

Skill	What good looks like
Problem breakdown	Split the task into small ordered steps (data model, function, validation, edge cases, tests).
Prompting	Give role, context, constraints, input/output examples and desired format.
Critical review	Read the generated code; spot bugs, security and complexity issues.
Testing	Ask for or write test cases, including edge cases; run them.
Iteration	Refine with precise feedback, not 'try again'.
Responsible use	No secrets or sensitive data in prompts; verify facts.

The 7-step workflow

- **1. Understand** the existing code and the requirement yourself first.
- **2. State the goal** in one sentence plus constraints (language, no new libraries, time complexity).
- **3. Ask for a plan** before code ('list the steps, no code yet').
- **4. Build in small pieces**: one function or component per prompt.
- **5. Review** each piece with the checklist below.
- **6. Test** with normal, edge and invalid inputs.
- **7. Refine and summarise** what changed.

Reusable prompt templates

PLAN: "I have a <language> project that <what it does>. I need to add <feature>. Constraints: <list>. First give a numbered plan of small steps. Do not write code yet."

BUILD: "Implement step 2 only: a function <name>(<params>) that <behaviour>. Input example: ... Expected output: ... Handle: empty input, null, duplicates. Return only the function with brief comments."

REVIEW: "Review this code for bugs, edge cases, time complexity and security. List issues in a table (issue, line, fix)."

TESTS: "Write 8 unit test cases for <function>: 3 normal, 3 edge, 2 invalid. Show input and expected output."

FIX: "Test 4 fails: input X gives Y, expected Z. Here is the code and the error. Explain the cause, then give a minimal fix."

Review checklist for AI-generated code

- Does it match the requirement exactly (not a similar problem)?
- Off-by-one, null/empty input, negative values, duplicates, large input?
- Time and space complexity acceptable?
- Does it modify existing code unnecessarily or break other features?
- Any hard-coded values, insecure input handling, SQL injection, unused imports?
- Are names, comments and structure consistent with the existing code base?

Worked mini-example: add a 'search by name' feature to a student list app

Step 1 prompt (plan):
"My Java console app stores Student(id, name, marks) in an ArrayList.
I need a case-insensitive search by name that supports partial matches.
Give a plan in 4-5 steps, no code yet."

Step 2 prompt (build):

```
"Write List<Student> searchByName(List<Student> list, String q).  
Case-insensitive, partial match, return empty list if q is null/blank.  
Do not change the Student class."
```

Your review: does it use toLowerCase(), contains(), null check? Is it O(n)?

Step 3 prompt (tests):

```
"Write 6 test cases: exact, partial, uppercase, no match, blank, null."
```

Step 4: run them, report failures back with input/expected/actual.

Common mistakes: pasting the whole problem and accepting the first answer; skipping review; never testing; vague follow-ups ('make it better'); not reading existing code before prompting.

9. Coding Round: Patterns and Solved Problems

Reported: two problems in about 45 minutes, test-case based scoring. Solve fully; partial test-case passes earn partial credit. Common topics: arrays, strings, hashing, two pointers, sorting, basic DP, matrix, number theory. Read input exactly as specified and watch edge cases. Solutions below are in Python; port to Java/C++ as needed.

1. Second largest distinct element ($O(n)$)

```
def second_largest(a):
    f = s = float('-inf')
    for x in a:
        if x > f: s, f = f, x
        elif f > x > s: s = x
    return None if s == float('-inf') else s
# [5, 5, 3] -> 3 | [2, 2] -> None
```

2. Reverse words / palindrome ignoring non-alphanumerics

```
def is_pal(s):
    t = [c.lower() for c in s if c.isalnum()]
    return t == t[::-1]
# "A man, a plan, a canal: Panama" -> True
```

3. Rotate array right by k

```
def rotate(a, k):
    n = len(a)
    if n == 0: return a
    k %= n
    return a[n-k:] + a[:n-k]
# [1,2,3,4,5], k=2 -> [4,5,1,2,3]
```

4. Two Sum (hash map, $O(n)$)

```
def two_sum(a, t):
    seen = {}
    for i, x in enumerate(a):
        if t - x in seen: return seen[t - x], i
        seen[x] = i
    return None
# [2,7,11,15], 9 -> (0, 1)
```

5. Longest substring without repeating characters (sliding window)

```
def longest_unique(s):
    last, start, best = {}, 0, 0
    for i, ch in enumerate(s):
        if ch in last and last[ch] >= start:
            start = last[ch] + 1
        last[ch] = i
        best = max(best, i - start + 1)
    return best
# "abcabcbb" -> 3
```

6. Valid parentheses (stack)

```
def valid(s):
    pair = {'(': ')', '[': ']', '{': '}'
    st = []
    for c in s:
        if c in '([{': st.append(c)
        elif c in pair:
            if not st or st.pop() != pair[c]: return False
    return not st
# "{[()]}" -> True | "]" -> False
```

7. Maximum subarray sum (Kadane, $O(n)$)

```
def kadane(a):
    cur = best = a[0]
    for x in a[1:]:
        cur = max(x, cur + x)
        best = max(best, cur)
    return best
# [-2,1,-3,4,-1,2,1,-5,4] -> 6
```

8. Primes up to n (Sieve)

```
def sieve(n):
    if n < 2: return []
    p = [True] * (n + 1); p[0] = p[1] = False
    for i in range(2, int(n ** 0.5) + 1):
        if p[i]:
            for j in range(i * i, n + 1, i): p[j] = False
    return [i for i in range(n + 1) if p[i]]
# sieve(20) -> [2,3,5,7,11,13,17,19]
```

9. Rotate matrix 90 degrees clockwise

```
def rotate_matrix(m):
    return [list(r) for r in zip(*m[::-1])]
# [[1,2],[3,4]] -> [[3,1],[4,2]]
```

10. Fibonacci with DP, and character frequency

```
def fib(n):
    if n < 2: return n
    a, b = 0, 1
    for _ in range(n - 1): a, b = b, a + b
    return b
# fib(10) -> 55
```

```
from collections import Counter
def first_non_repeating(s):
    c = Counter(s)
    for ch in s:
        if c[ch] == 1: return ch
    return None
# "swiss" -> 'w'
```

More topics to practise (pick 2-3 per day)

- Arrays: move zeros, merge two sorted arrays, majority element, leaders, prefix sums.
- Strings: anagram check, count vowels/consonants, compress string (aaabb to a3b2), longest common prefix.
- Number theory: GCD/LCM, Armstrong, perfect number, digit sum, base conversion.
- Sorting and searching: custom sort, binary search variants, kth largest.
- Patterns: pyramid, diamond, Pascal triangle.
- Basic DP: climbing stairs, coin change (min coins), 0/1 knapsack, LIS.

Contest-day coding tips

- Write input handling first and test the sample.
- Think of brute force, then optimise if constraints are large (n up to 10^5 needs $O(n \log n)$ or better).
- Use long for large sums; avoid recomputation; test with empty, single element, all equal, negatives.
- Submit a correct brute force rather than leave a problem unsolved.

10. English Communication

About 30 questions: grammar, vocabulary, reading comprehension, sentence completion, error spotting. Timed, so build speed.

High-frequency grammar rules

- **Subject-verb agreement:** 'Each of the students has' (each, every, either, neither take singular). 'The number of' is singular, 'A number of' is plural.
- **Tenses:** use present perfect for unfinished or recent actions ('I have lived here for 5 years'); simple past with a finished time ('I lived there in 2020').
- **Articles:** a/an by sound ('an hour', 'a university'); 'the' for specific or unique things.
- **Prepositions:** interested in, good at, afraid of, depend on, different from, capable of, congratulate on, prefer X to Y.
- **Conditionals:** If + present, will; If + past, would; If + past perfect, would have.
- **Parallel structure:** 'He likes reading, writing and to swim' is wrong; use 'swimming'.
- **Commonly confused:** affect/effect, their/there, its/it's, principal/principle, lose/loose, then/than.
- **Active/passive, direct/indirect speech:** shift tense back one step in indirect speech.

Practice

Q39. Choose the correct sentence:

- (a) Neither of the answers are correct.
- (b) Neither of the answers is correct.
- (c) Neither of the answer is correct.
- (d) Neither of answers is correct.

Answer: (b) | 'Neither' takes a singular verb.

Q40. She is good ____ mathematics.

- (a) in
- (b) at
- (c) on
- (d) for

Answer: (b) | good at.

Q41. Synonym of 'meticulous':

- (a) Careless
- (b) Thorough
- (c) Rapid
- (d) Rude

Answer: (b) | Meticulous = very careful and precise.

Q42. Antonym of 'scarce':

- (a) Rare
- (b) Plentiful
- (c) Costly
- (d) Tiny

Answer: (b) | Scarce = insufficient; opposite is plentiful.

Q43. If I ____ you, I would apologise.

- (a) am
- (b) was
- (c) were
- (d) be

Answer: (c) | Subjunctive in unreal conditionals.

Q44. Identify the error: 'The team have won its match yesterday.'

- (a) The team have
- (b) won its
- (c) yesterday
- (d) No error

Answer: (a) | Should be 'The team has' for a collective noun as a unit (with 'its').

Reading comprehension strategy

- Read the questions first, then skim the passage for keywords.
- Eliminate extreme options (always, never, only) unless the passage says so.
- Answer from the passage, not from outside knowledge.

11. Cognitive Games and Behavioural Assessment

Cognitive (game-based)

Reported as about 4 short interactive games measuring attention, memory, pattern recognition and logical reasoning. The exact games change between drives, so prepare the underlying skills:

- **Pattern recognition:** practise number series, figure series and odd-one-out.
- **Working memory:** sequence recall (digits, positions) and n-back style tasks.
- **Attention and speed:** respond accurately and quickly; do not sacrifice accuracy for speed.
- **Risk/decision tasks:** be consistent; there is no 'trick'. Read instructions carefully, do the practice round.
- Environment: quiet room, laptop plugged in, mouse if allowed, no notifications, good sleep.

Behavioural / power-skills questionnaire

- Answer **honestly and consistently**; similar questions are repeated with different wording to check consistency.
- Typical traits: teamwork, adaptability, ownership, learning agility, communication, resilience.
- Choose the answer that reflects how you really behave at work or college, in a balanced way (neither extreme).
- Do not overthink; first honest instinct is usually best.

12. HR and Technical Interview Questions

HR

- **Tell me about yourself:** 60-90 seconds: name, degree and college, key skills, one project, one achievement, why Capgemini.
- **Why Capgemini?** Global consulting and technology services, wide range of domains, structured learning, exposure to cloud, AI and digital engineering.
- **Strengths and weaknesses:** give real strength with an example; weakness with the step you are taking to improve.
- **Are you willing to relocate or work in shifts?** Answer honestly.
- **Where do you see yourself in 5 years?** Skilled engineer growing toward technical ownership.
- **Conflict in a team / failure / tight deadline:** use STAR (Situation, Task, Action, Result).

Technical

- Explain your final-year project: problem, tech stack, your role, a bug you fixed, what you would improve.
- OOP pillars with real examples; difference between abstract class and interface; method overloading vs overriding.
- Difference between array and linked list; stack vs queue; HashMap internals; time complexity of common operations.
- SQL: joins, group by, second highest salary, primary vs foreign key, normalisation.
- Basics of cloud (IaaS/PaaS/SaaS), REST API, HTTP methods (GET, POST, PUT, DELETE), Git workflow.
- How do you use AI tools while coding? Answer: for ideas, boilerplate and tests, with careful review; never share confidential data; I stay accountable for the code.
- Write a short program live: reverse a string, check palindrome, find duplicates, FizzBuzz.

Resume rules

- Be able to explain every line, every technology and every project on your resume.
- Keep projects concrete: problem, approach, result with a number if possible.

13. Last-48-Hours Checklist

- Confirm date, time, platform link and system requirements from the invite email.
- Test camera, microphone, internet and charger; close all other apps and tabs.
- Keep a valid photo ID ready; sit in a quiet, well-lit room.
- Revise only: port table, complexity table, bug catalogue, prompt templates, error log.
- Sleep 7+ hours; eat light; reach the test link 15 minutes early.
- In the test: attempt every MCQ, manage time per section, do not panic if one section feels hard.

Sources used for the pattern

- FACE Prep: Capgemini Exceller assessment framework 2026, technical assessment (AI Literacy), debugging assessment, AI-assisted coding round, previous year questions guides (faceprep.in).
- Placement Preparation: Capgemini syllabus and test pattern 2026; preparation tips (placementpreparation.io).
- GUVI: Capgemini Exceller syllabus 2026 and recruitment process guides (guvi.in).
- Knowledge Gate, LinkedIn and Reddit candidate reports on the 2027 pattern (as shared in your notes).

Patterns are reported by candidates and prep sites and vary by campus, role and drive. Always follow the official instructions in your Capgemini invite.